

A8 · Answer Key and Demo Script

CS425 · TCP on a Noisy Link · instructor led

INSTRUCTOR COPY

Every transcript below is real output captured on `onyx.boisestate.edu` on October 4, 2026. The loss is random, so the numbers move from run to run, but the shape does not. About 45 minutes: 10, 10, 10, 11, then 4 for the exit question.

Say this at the start

"Chapter 7 says TCP does badly on wireless links because it thinks noise is congestion. Onyx has no radio, so we are going to build a bad link out of software and measure how badly. Everyone `ssh onyx` and grab the meter. Predict before we run anything." Then hand out the sheet.

ROUND 1 · BUILD A SLOW LINK

`unshare -rn bash · ip link set lo up mtu 1500 · ping, then tc, then ping again`

```
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.050 ms
qdisc netem 8004: root refcnt 2 limit 1000 delay 25ms rate 20Mbit seed 8230483813303860672
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=50.2 ms
```

SAY "Inside this shell you are root of a network with one interface in it. Nobody else on Onyx can see it, and it is gone when you exit." Run `ip -br link` to prove there is only `lo`.

RTT. 0.05 ms before, 50.2 ms after. Most of the room predicts 25.

Q1. The echo request leaves through `lo` and is held 25 ms, then the reply leaves through `lo` and is held another 25 ms. One interface plays both directions of the link.

Transmission delay. $1,500 \times 8 = 12,000$ bits / $20,000,000$ bit/s = 0.6 ms. Small next to 25 ms of propagation, which is why the ping barely notices the rate.

ROUND 2 · A CLEAN LINK

`python3 tcpmeter.py`

sec	Mbit/s	cwnd	ssthresh	rtt ms	retrans
1	12.37	128	80	95.3	0
2	16.02	191	80	127.5	0
5	18.22	427	80	256.2	0
10	19.85	823	80	493.8	0

average 17.72 Mbit/s over 10 s, MSS 1448 bytes, 0 segments retransmitted

POINT AT the `rtt` column: the link has a 50 ms RTT and TCP is measuring ten times that.

Bandwidth-delay product. $20,000,000 \times 0.05 = 1,000,000$ bits = 125,000 bytes, and $125,000 / 1,448 \approx 86$ segments.

Q2. In a queue. Everything past 86 segments waits in `netem`'s buffer, the same as a router's output queue. $823 - 86 = 737$ segments $\times 1,448 \times 8 \approx 8.5$ Mbit, which takes 0.43 s to drain at 20 Mbit/s. Add the 50 ms base and you get about 480 ms, against 494 measured. CUBIC only backs off on loss, so it keeps filling the buffer until it overflows. That is bufferbloat.

FAST GROUP Why is `ssthresh` 80 with no loss? HyStart: CUBIC leaves slow start early when it sees the RTT climbing, and sets `ssthresh` to the window at that moment.

GRADING

Pass/fail, 20 points. Pass when every box has something in it and the round 2 to 4 arithmetic was attempted. Wrong predictions are fine; that is what they are for.

ROUND 3 · ADD NOISE

tc qdisc change dev lo root netem delay 25ms rate 20mbit loss 2% · python3 tcpmeter.py					
sec	Mbit/s	cwnd	sssthresh	rtt ms	retrans
1	7.46	21	19	51.5	13
2	3.39	10	9	54.2	21
5	2.07	10	8	53.3	32
10	1.59	4	3	62.3	53
average 2.70 Mbit/s over 10 s, MSS 1448 bytes, 53 segments retransmitted					
SAY "Same link, still 20 Mbit/s, nothing full. TCP lost one segment in fifty and threw away 85% of the link."					

Formula. $MSS = 1,448 \times 8 = 11,584$ bits. $11,584 / 0.05 = 231,680$ bit/s. $\sqrt{0.02} = 0.141$. $1.22 \times 231,680 / 0.141 \approx \mathbf{2.0}$ Mbit/s. Measured 2.7: the formula models Reno, and CUBIC with SACK does a little better. Same ballpark is the point.

Q3. TCP cannot tell noise from congestion. Every loss looks like a full router queue, so it cuts the window (CUBIC to 70%, Reno to half). At 2% a loss comes every 50 segments or so, and the window sits around 10, nowhere near the 86 it needs. Section 7.6: it slows down for the wrong reason.

ROUND 4 · WHAT 802.11 DOES ABOUT IT

Lost after 7 tries. $0.2^7 = 0.0000128$, about **0.0013%**.

Average tries. $1 / (1 - 0.2) = 1.25$, so $20 / 1.25 = \mathbf{16}$ Mbit/s left for new data. 20% loss per try costs 20% of the bandwidth, not 90% of it.

TCP's throughput. No retries: $1.22 \times 231,680 / \sqrt{0.2} \approx \mathbf{0.63}$ Mbit/s. With retries the formula gives about 78 Mbit/s, more than the link has, so the link is the limit: **about 16**.

loss 20%	then	rate 16mbit	loss 0.0013%
average 0.36 Mbit/s over 10 s, MSS 1448 bytes, 62 segments retransmitted			
average 14.31 Mbit/s over 10 s, MSS 1448 bytes, 0 segments retransmitted			
SAY "Forty times faster, same radio. The only difference is which layer fixed the loss."			

Q4. First run: TCP recovered every loss end to end, and read each one as congestion. Second run: the link layer recovered it with 802.11's ACK and retransmit, one hop, and TCP never saw it. The second run is local recovery. 0.36 is under the formula's 0.63 because at 20% loss timeouts dominate, and the formula assumes none.

EXIT QUESTION

A radio's own transmission is far stronger at its antenna than anything it could receive, so it cannot listen while it sends. With hidden terminals the collision happens at the access point anyway, where the sender could never hear it. So 802.11 avoids collisions instead of detecting them (CSMA/CA): sense the channel, wait DIFS, back off a random time that counts down only while the channel is idle, send the whole frame, and wait for an ACK. No ACK means retransmit with a bigger backoff. Optional RTS/CTS: the AP's CTS reaches the hidden terminal, which stays quiet for the reserved time.